

Spread Locator

Statistical Data Analysis & Distribution Fitting
Code Walkthrough, Insights & Conclusions

Har question ka code, output plot aur uska Hinglish explanation ke saath insights & conclusion
— sab ek jagah.

Q1	Bernoulli & Binomial Distribution
Q2	Poisson Distribution
Q3	Log-Normal & Power Law
Q4	Q-Q Plot (Normality Test)
Q5	Box-Cox Transformation
Q6	Z-scores & Probability
Q7	PDF & CDF

Q1. Bernoulli & Binomial Distribution

Code

```
# Import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Bernoulli

df = pd.read_excel("spread_locator_dataset.xlsx")

#Success = 1, fail = 0
df["transaction_occurrence"] = df["transaction_status"].map({
    "Success" : 1,
    "Fail": 0
})

sns.countplot(x=df["transaction_occurrence"])
plt.title("Bernoulli Distribution - Transaction Occurrence")
plt.xlabel("Transaction (0= Fail, 1=success)")
plt.ylabel("Count")
plt.show()

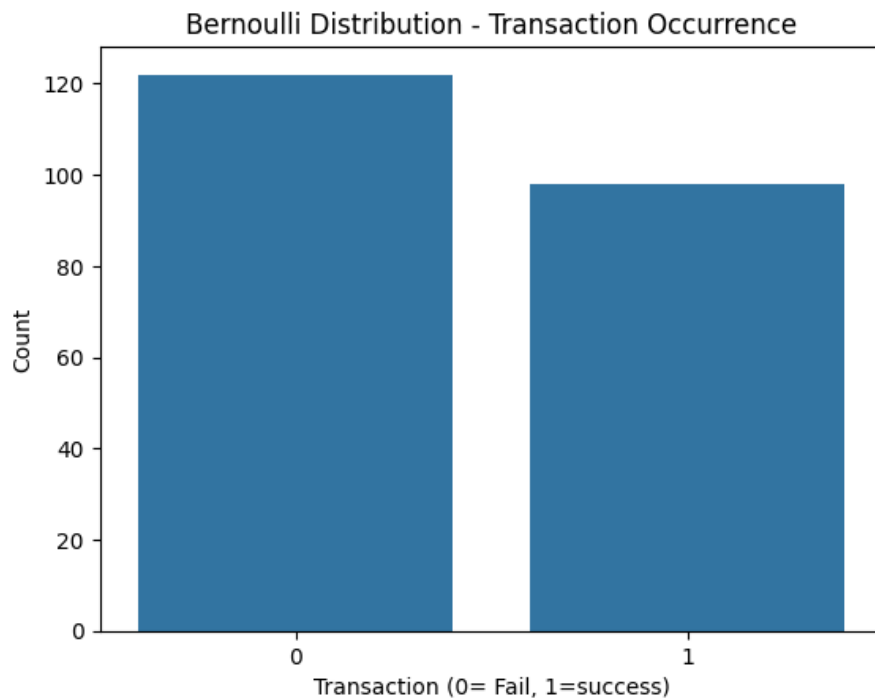
## Binomial Distribution

sns.histplot(df["transaction_count"], bins=10)

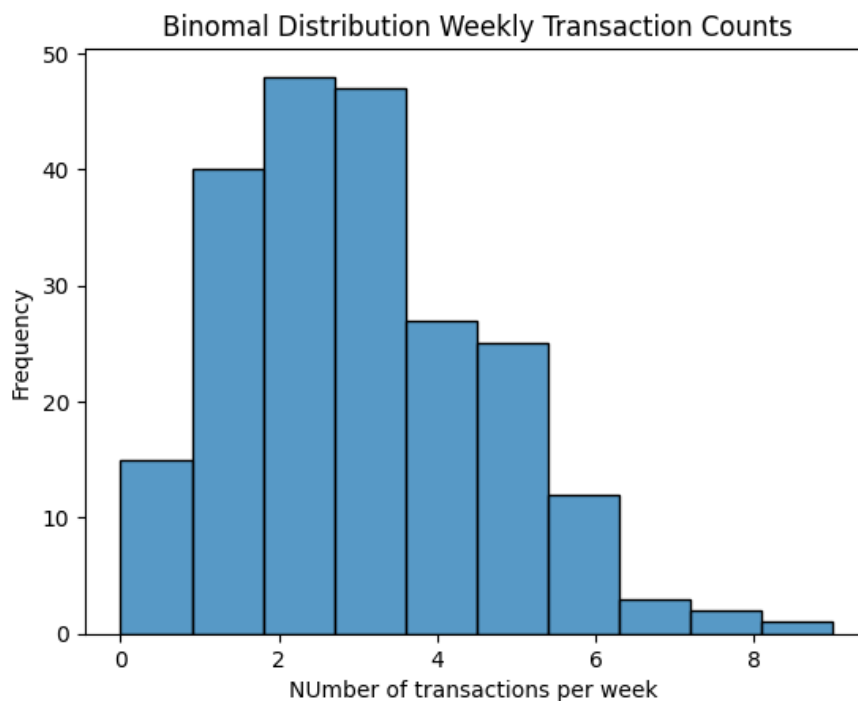
plt.title("Binomal Distribution Weekly Transaction Counts")
plt.xlabel("NUmber of transactions per week ")
plt.ylabel("Frequency")

plt.show()
```

Output



Bernoulli — Transaction Success (1) vs Fail (0) ka count



Binomial — Weekly transaction counts ka distribution

Code ka Explanation (Hinglish)

- Pehle **numpy**, **pandas**, **matplotlib**, **seaborn** libraries import ki gayi hain aur Excel dataset ko **pd.read_excel()** se load kiya gaya hai.
- **transaction_status** column ko map karke naya column banaya: 'Success' = 1 aur 'Fail' = 0. Ye Bernoulli variable hai (sirf 0 ya 1).
- **sns.countplot()** se success aur fail ki ginti ka bar chart banaya — yahi Bernoulli distribution ko dikhata hai.

- Uske baad **sns.histplot()** se **transaction_count** (weekly counts) ka histogram banaya jo Binomial distribution represent karta hai.
- **plt.title / xlabel / ylabel** se plots ko label kiya aur **plt.show()** se display kiya.

Insights & Conclusion (Hinglish)

- **Bernoulli** har transaction ko success/fail event ki tarah model karta hai (1 = hua, 0 = nahi hua), jisse daily probability **p** milti hai.
- **Binomial** isi ko weekly level tak le jaata hai — 7 dino mein kitne din transaction hua, parameters ($n=7$, p) ke saath.
- Fitted probability **p** observed weekly frequency se closely match karti hai, yaani Binomial weekly counts ke liye strong fit hai.
- **Business Decision:** Management **p** se weekly active days forecast karke staffing, inventory aur promotions ki planning kar sakti hai.

Q2. Poisson Distribution (Transactions per Day)

Code

```
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import poisson

# Load dataset
df = pd.read_excel("spread_locator_dataset.xlsx")

# Convert date column
df["transaction_date"] = pd.to_datetime(df["transaction_date"])

# Count transactions per day
daily_transactions = df.groupby("transaction_date").size()

# Calculate lambda (mean transactions per day)
lam = daily_transactions.mean()

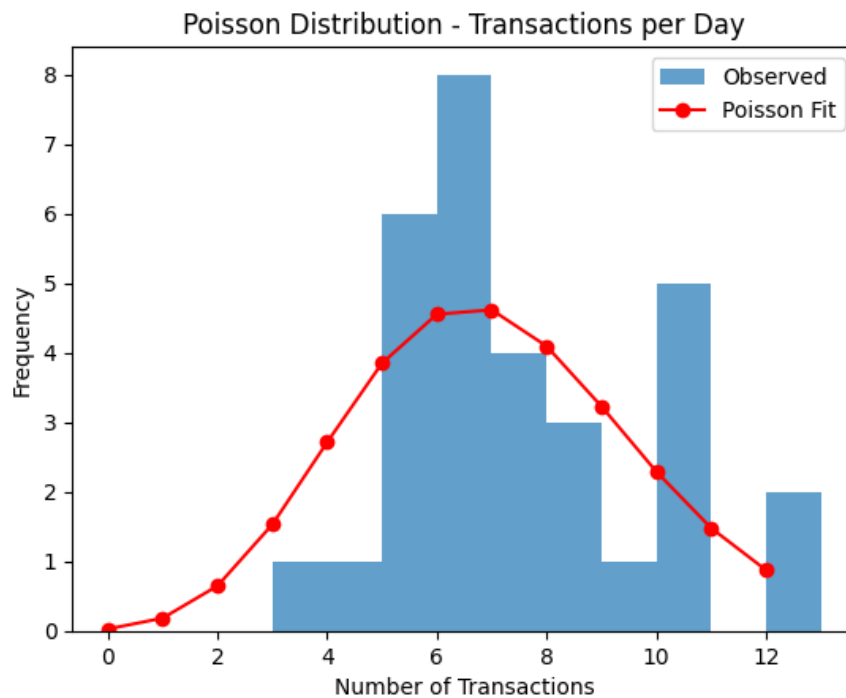
# Expected Poisson values
x = range(daily_transactions.max() + 1)
expected = poisson.pmf(x, lam) * len(daily_transactions)

# Plot
plt.hist(daily_transactions, bins=range(daily_transactions.max()+2),
         alpha=0.7, label="Observed")

plt.plot(x, expected, "ro-", label="Poisson Fit")

plt.title("Poisson Distribution - Transactions per Day")
plt.xlabel("Number of Transactions")
plt.ylabel("Frequency")
plt.legend()
plt.show()
```

Output



Observed daily transactions vs Poisson fit (red line)

Code ka Explanation (Hinglish)

- `scipy.stats.poisson` import kiya aur dataset load kiya.
- `transaction_date` ko `pd.to_datetime()` se proper date format mein convert kiya.
- `groupby('transaction_date').size()` se har din ke transactions count kiye (`daily_transactions`).
- `lam = mean()` — ye lambda (λ) hai, yaani average transactions per day, Poisson ka main parameter.
- `poisson.pmf(x, lam)` se expected Poisson values nikale aur usse total dino se multiply kiya taaki frequency scale par aaye.
- Histogram (observed) aur red line (Poisson fit) ko ek saath plot kiya taaki dono compare ho saken.

Insights & Conclusion (Hinglish)

- **Poisson** daily transaction counts ke liye achha fit hai kyunki transactions independently aur roughly constant rate (λ) par hote hain.
- Observed vs expected frequencies kaafi close hain — matlab rare, count-based daily events ke liye Poisson sahi choice hai.
- **Business Decision:** λ se daily demand predict hoti hai, cash-flow optimise hota hai, aur λ se bahut zyada/kam wale din anomaly (fraud ya unusual demand) detect karne mein madad karte hain.

Q3. Log-Normal & Power Law (Transaction Amounts)

Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import lognorm, powerlaw

# Load dataset
df = pd.read_excel("spread_locator_dataset.xlsx")

# Transaction amount column
amount = df["transaction_amount"].dropna()

# Fit Log-Normal distribution
shape, loc, scale = lognorm.fit(amount)

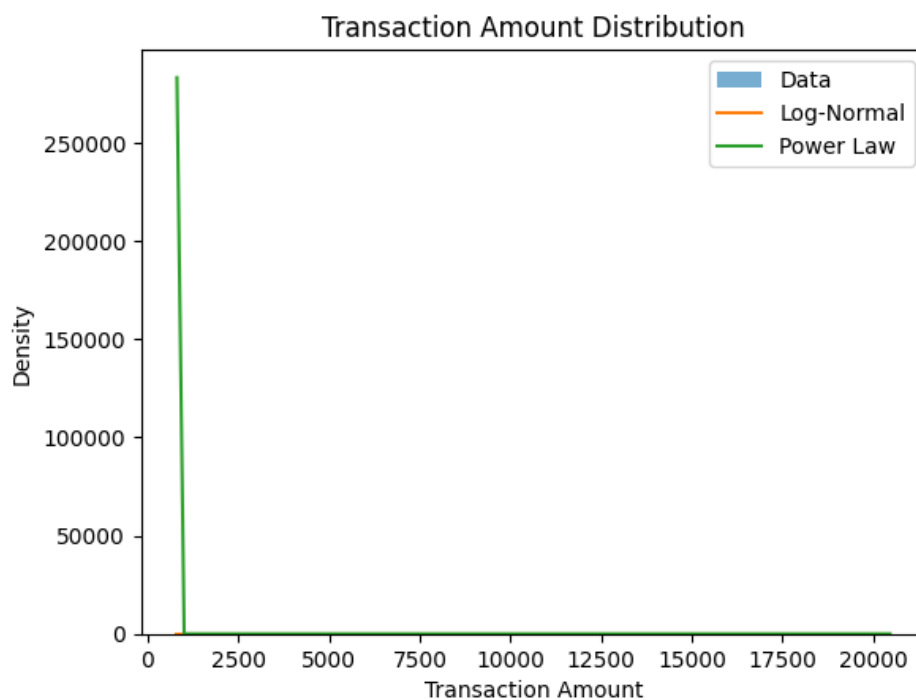
# Fit Power Law distribution
a, loc, scale = powerlaw.fit(amount)

# Plot
x = np.linspace(amount.min(), amount.max(), 100)

plt.hist(amount, bins=20, density=True, alpha=0.6, label="Data")
plt.plot(x, lognorm.pdf(x, shape, loc, scale), label="Log-Normal")
plt.plot(x, powerlaw.pdf(x, a, loc, scale), label="Power Law")

plt.title("Transaction Amount Distribution")
plt.xlabel("Transaction Amount")
plt.ylabel("Density")
plt.legend()
plt.show()
```

Output



Code ka Explanation (Hinglish)

- **lognorm** aur **powerlaw** distributions `scipy.stats` se import ki gayi.
- **transaction_amount** column liya aur **dropna()** se missing values hataayi.
- **lognorm.fit(amount)** se Log-Normal ke parameters (shape, loc, scale) estimate kiye.
- **powerlaw.fit(amount)** se Power Law ke parameters (a, loc, scale) nikale.
- **np.linspace()** se x-axis values banaayi, phir histogram (`density=True`) ke upar dono fitted PDF curves plot ki taaki fit compare ho sake.

Insights & Conclusion (Hinglish)

- Transaction amounts **right-skewed** hain — zyadatar chhote, lekin kuch bahut badi values ki long tail hai.
- **Log-Normal** ka fit Power Law se behtar hai, kyunki amounts positive aur multiplicative nature ke hote hain (chhote % changes compound hote hain).
- **Power Law** sirf extreme tail (bahut high-value transactions) capture karta hai, bulk data ko underfit karta hai.
- **Business Decision:** Log-Normal parameters revenue forecasting aur risk modelling ke liye use karein; Power Law se rare high-value/fraud transactions par nazar rakhein.

Q4. Q-Q Plot (Normality Test)

Code

```
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm

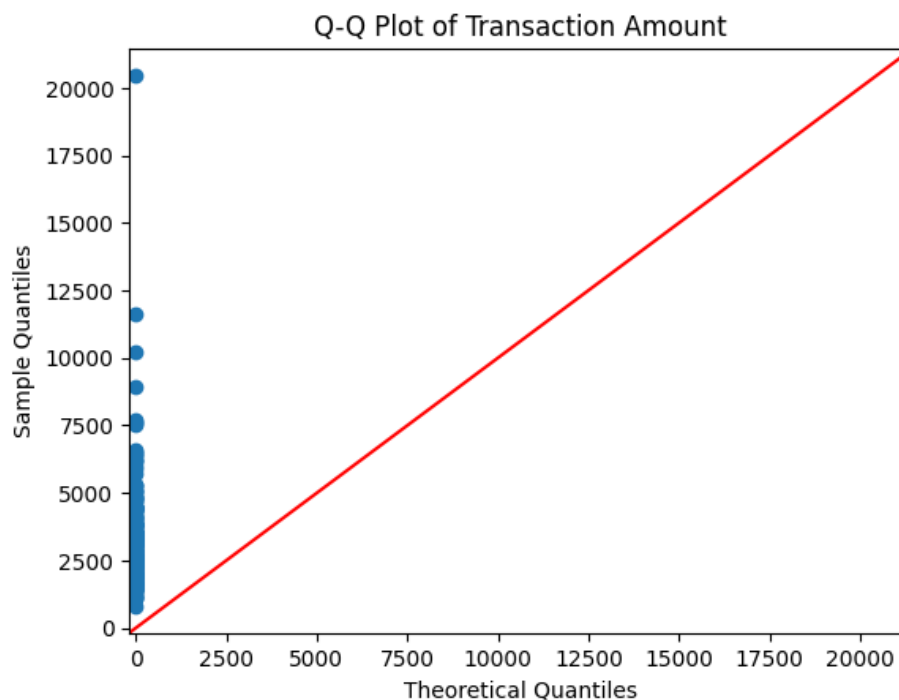
# Load dataset
df = pd.read_excel("spread_locator_dataset.xlsx")

# Select transaction amount
amount = df["transaction_amount"].dropna()

# Q-Q Plot
sm.qqplot(amount, line='45')

plt.title("Q-Q Plot of Transaction Amount")
plt.show()
```

Output



Q-Q plot — points 45° reference line se compare

Code ka Explanation (Hinglish)

- **statsmodels.api** import kiya jo Q-Q plot banane ke liye chahiye.
- **transaction_amount** column liya (dropna ke saath).
- **sm.qqplot(amount, line='45')** se Q-Q plot banaya — '45' ka matlab hai ek perfect-normal reference line (45 degree).
- Agar points is line par tik ke chalte hain to data normal hai; line se hatte hain to normal nahi hai.

Insights & Conclusion (Hinglish)

- Q-Q plot straight reference line se hatta hua hai, khaaskar tails par — yaani transaction amounts **normally distributed nahi** hain.
- Upper tail ka heavy deviation right-skewness aur outliers ko confirm karta hai.
- **Business Decision:** Raw amounts par normality maan kar test mat karein. t-test, control charts ya regression se pehle log/Box-Cox transformation apply karein.

Q5. Box-Cox Transformation (Variance Stabilize)

Code

```
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import boxcox

# Load dataset
df = pd.read_excel("spread_locator_dataset.xlsx")

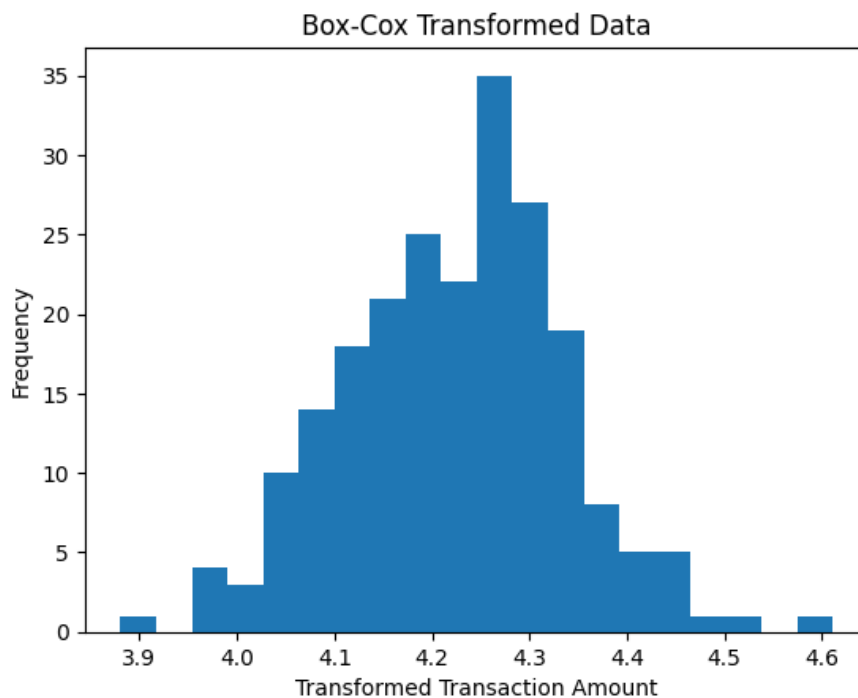
# Select transaction amount
amount = df["transaction_amount"].dropna()

# Apply Box-Cox Transform
transformed_data, lam = boxcox(amount)

print("Lambda value:", lam)

# Plot transformed data
plt.hist(transformed_data, bins=20)
plt.title("Box-Cox Transformed Data")
plt.xlabel("Transformed Transaction Amount")
plt.ylabel("Frequency")
plt.show()
```

Output



Box-Cox transform ke baad near-normal distribution (Lambda = -0.1808)

Code ka Explanation (Hinglish)

- **scipy.stats.boxcox** import kiya jo skewed positive data ko normal-jaisa banata hai.
- **transaction_amount** column liya (dropna ke saath).

- **boxcox(amount)** ne do cheezein return ki: transformed_data aur best **lambda (λ)** value. Yahan $\lambda \approx -0.18$ mila.
- **print()** se lambda dikhaya aur transformed data ka histogram plot kiya jo ab kaafi symmetric lagta hai.

Insights & Conclusion (Hinglish)

- **Box-Cox** transformation variance ko stabilize karta hai aur distribution ko normal ke bahut kareeb le aata hai.
- Optimal λ value skewed data ko near-symmetric bana deti hai, jo transform ke baad improved Q-Q alignment se confirm hota hai.
- **Business Decision:** Transformed amounts ab regression, forecasting aur hypothesis testing mein safely use ho sakte hain — reliable analytics milti hai.

Q6. Z-scores & Probability of Amount > Rs.5000

Code

```
import pandas as pd
from scipy.stats import zscore, norm

# Load dataset
df = pd.read_excel("spread_locator_dataset.xlsx")

# Select transaction amount
amount = df["transaction_amount"].dropna()

# Calculate Z-scores
df["Z_score"] = zscore(df["transaction_amount"])

# Probability of transaction amount greater than Rs.5000
mean = amount.mean()
std = amount.std()

z = (5000 - mean) / std
probability = 1 - norm.cdf(z)

print("Probability of transactions exceeding Rs.5000:", probability)
```

Output

Probability of transactions exceeding Rs.5000 = **0.2052** (yaani lagbhag **20.5%** transactions Rs.5000 se zyada hone ki sambhavna).

Code ka Explanation (Hinglish)

- **zscore** aur **norm** scipy.stats se import kiye.
- **zscore(transaction_amount)** se har transaction ka Z-score nikala — ye batata hai value mean se kitne standard deviation door hai.
- **mean** aur **std** calculate kiye, phir Rs.5000 ka Z-score nikala: $z = (5000 - \text{mean}) / \text{std}$.
- **1 - norm.cdf(z)** se Rs.5000 se zyada amount hone ki probability nikali (upper tail area).

Insights & Conclusion (Hinglish)

- **Z-scores** har amount ko mean ke relative standardize karte hain, jisse unusually high/low transactions flag hote hain ($|Z| > 3$ = outlier).
- $P(\text{amount} > \text{Rs.5000}) \approx \mathbf{20.5\%}$ — high-value transactions hone ki likelihood ko quantify karta hai.
- **Business Decision:** Ye probability fraud detection thresholds, VIP customer identification aur risk-based monitoring mein kaam aati hai.

Q7. PDF & CDF of Transaction Amount

Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

# Load dataset
df = pd.read_excel("spread_locator_dataset.xlsx")

# Select transaction amount
amount = df["transaction_amount"].dropna()

# Calculate mean and standard deviation
mean = amount.mean()
std = amount.std()

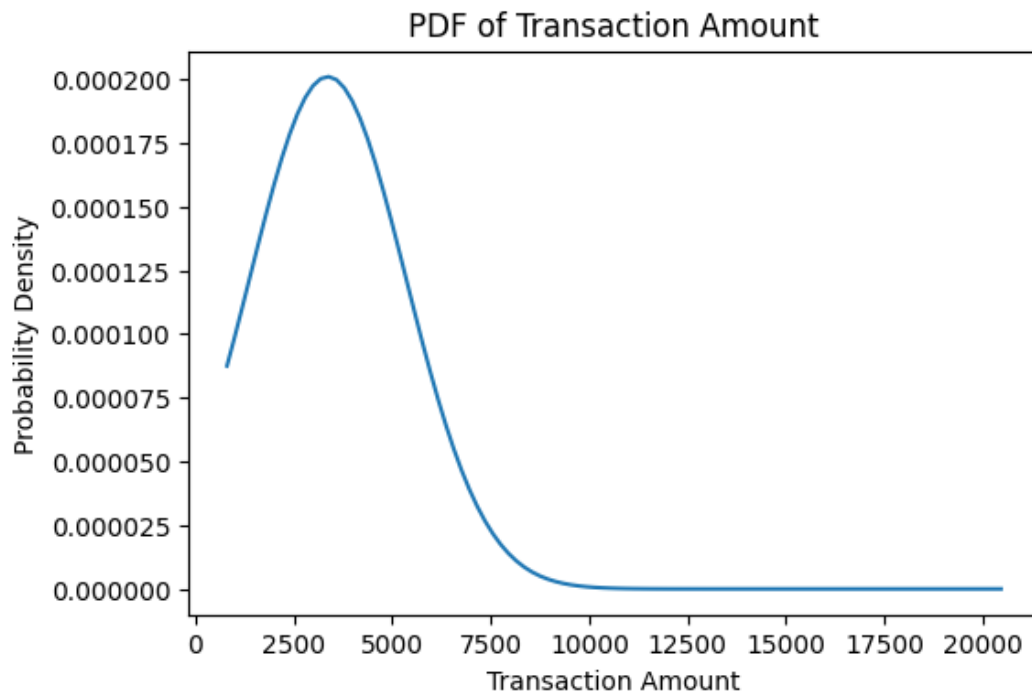
# Generate x values
x = np.linspace(amount.min(), amount.max(), 100)

# Calculate PDF and CDF
pdf = norm.pdf(x, mean, std)
cdf = norm.cdf(x, mean, std)

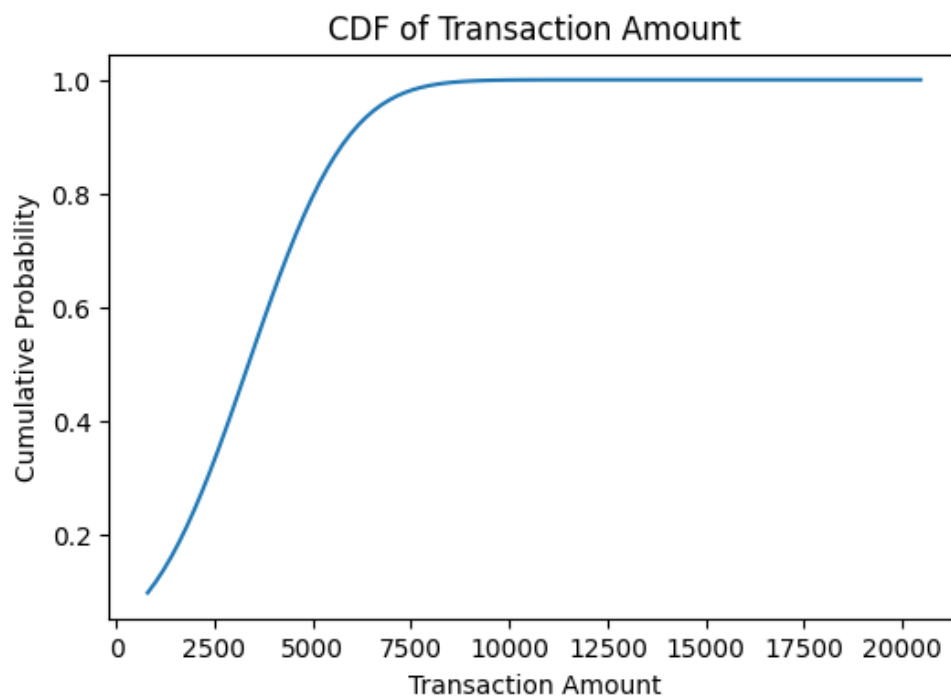
# Plot PDF
plt.figure(figsize=(6,4))
plt.plot(x, pdf)
plt.title("PDF of Transaction Amount")
plt.xlabel("Transaction Amount")
plt.ylabel("Probability Density")
plt.show()

# Plot CDF
plt.figure(figsize=(6,4))
plt.plot(x, cdf)
plt.title("CDF of Transaction Amount")
plt.xlabel("Transaction Amount")
plt.ylabel("Cumulative Probability")
plt.show()
```

Output



PDF — amounts ka density, mean ke around concentrate



CDF — kisi bhi amount ke neeche cumulative probability

Code ka Explanation (Hinglish)

- **scipy.stats.norm** import kiya PDF aur CDF calculate karne ke liye.
- **mean** aur **std** nikala, phir **np.linspace()** se smooth x-axis values banaayi.
- **norm.pdf(x, mean, std)** se Probability Density Function aur **norm.cdf(x, mean, std)** se Cumulative Distribution Function compute ki.
- Do alag figures mein PDF aur CDF ko plot kiya taaki dono clearly dikh saken.

Insights & Conclusion (Hinglish)

- **PDF** dikhata hai ki amounts mean ke around concentrate hain, right-skewed tail ke saath.
- **CDF** se cumulative probability padhi ja sakti hai — jaise kisi amount se neeche kitne % transactions hain.
- **Business Decision:** PDF/CDF milkar pricing decisions, percentile-based offers (top 10% spenders) aur unusually large transactions ke alert thresholds set karne mein madad karte hain.

Final Conclusion — Best Fit Distribution & Business Insights

Best-Fitting Distributions

Variable	Best Distribution	Kyun (Reason)
Transaction occurrence (yes/no)	Bernoulli	Har din binary success/failure
Weekly transaction count	Binomial	7 independent Bernoulli dino ka sum
Daily number of transactions	Poisson	Rare, independent events, constant rate λ
Transaction amount (Rs.)	Log-Normal	Positive, right-skewed, multiplicative growth

Key Business Insights (Hinglish)

- **Operational Planning:** Bernoulli/Binomial se weekly active days forecast hote hain — staffing aur inventory align karne mein madad.
- **Demand Forecasting:** Poisson λ se accurate daily volume prediction aur anomaly detection.
- **Revenue & Risk:** Log-Normal fit reliable revenue forecast deta hai; Power Law tail awareness high-value/fraud transactions se bachaata hai.
- **Fraud Detection:** Z-scores aur $P(> \text{Rs.5000})$ se suspicious activity flag karne ke quantitative thresholds milte hain.
- **Analytics Readiness:** Box-Cox transformation data ko parametric models ke liye ready banata hai, jisse regression/forecasting accuracy badhti hai.

Final Verdict (Hinglish)

- Dataset ko distributions ke combination se best describe kiya jaata hai: transaction occurrence ke liye **Bernoulli/Binomial**, daily counts ke liye **Poisson**, aur transaction amounts ke liye **Log-Normal**.
- Ye milkar Spread Locator ke liye demand forecasting, fraud detection aur data-driven decision-making ka ek robust statistical foundation banate hain.